VELLORE INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FRAUDLENS

Financial Transaction and Fraud Investigation Management System

A COMPREHENSIVE DATABASE MANAGEMENT SYSTEMS CASE STUDY REPORT

Institution:	Vellore Institute of Technology
Submitted By:	1. D Joshua Daniel (Reg No: 24BCE0436) 2. M Saryu (Reg No: 24BCE2524)
Degree & Branch:	Bachelor of Technology in Computer Science and Engineering
Subject:	Database Management Systems
Faculty Evaluator:	Prof. Kanagaraj K
Database Platform:	Oracle AI Database 26ai Free Release 23.26.3.0.0
PDB & Schema:	FREEPDB1 / FRAUDLENS_USER
Host Platform:	Microsoft Windows 11 Home Single Language | 8.0 GB RAM | Core i3-1115G4

Submission Date: *October 2026 | Evaluated via Native Oracle SQL*Plus Engine*

Table of Contents

Section Title	Page
1. Cover Page	1
2. Abstract	3
3. Introduction	3
4. Problem Statement	3
5. Objectives of the Project	3
6. Existing System and Its Limitations	4
7. Proposed System — FraudLens	4
8. Scope of the Project	4
9. Software and Hardware Requirements	4
10. System Architecture	5
11. Entity-Relationship (ER) Diagram	5
12. Relational Schema	6
13. Database Design and Normalization (1NF, 2NF, 3NF)	7
14. Table Structures and Relationships	7
15. Business Rules and Constraints	11
16. SQL Implementation	12
17. DDL and DML Operations	12
18. SQL Queries, Joins, Subqueries, and Aggregate Functions	12
19. Database Views	13
20. Stored Procedures and Functions	13
21. Database Cursors	13
22. Database Triggers and Audit Management	14
23. Fraud Detection and Alert Management Logic	14
24. Investigation Management and Workflow	14
25. Testing Methodology and Test Cases	14
26. Actual Execution Screenshots and Sample Outputs	16
27. Results and Discussion	19
28. Advantages and Limitations	19
29. Future Enhancements	19
30. Conclusion	19
31. References	20

2. Abstract

Modern digital banking platforms process tens of millions of financial transactions daily across Unified Payments Interface (UPI), Internet Net-Banking, Mobile Applications, Point-of-Sale (POS), and Automated Teller Machines (ATMs). Ensuring low-latency payment execution while simultaneously maintaining rigorous Anti-Money Laundering (AML) surveillance presents a fundamental database design challenge. **FraudLens: Financial Transaction and Fraud Investigation Management System** is a comprehensive academic database case study developed for B.Tech Computer Science and Engineering, demonstrating how enterprise relational database principles solve these dual requirements.

A foundational principle of FraudLens is the explicit decoupling of automated risk signals from legal proof of fraud: **a fraud alert indicates an anomaly requiring review; it does not prove criminal culpability**. Implemented on Oracle AI Database 26ai Free Release 23.26.3.0.0 (container FREEPDB1, schema FRAUDLENS_USER), FraudLens organizes data into 8 relational tables normalized to Third Normal Form (3NF), 8 synthetic primary key sequences, 4 analytical surveillance views, 3 PL/SQL stored procedures, 3 stored functions, 2 row-level audit triggers, and 71 data-dictionary integrity constraints. An automated 13-case test harness verified the expected operational workflows and constraint rejections across the documented execution run with a 13/13 (100%) recorded pass rate, validating baseline functional integrity without claiming exhaustive verification.

Academic Classification & Scope: FraudLens is designed as an educational, auditable relational prototype demonstrating ACID transactional consistency, declarative constraint enforcement, and in-database procedural automation. It does not claim enterprise production-grade scalability or external regulatory compliance, but provides a fully reproducible reference architecture for academic evaluation.

3. Introduction

The digital transformation of retail banking has replaced batch-clearing systems with round-the-clock instant payment rails. In India, platforms like UPI handle billions of monthly transactions, demanding sub-second ledger updates. Concurrently, financial institutions face sophisticated cyber threats including automated account takeover, rapid velocity cash draining, smurfing (structuring transfers below regulatory thresholds), and unauthorized cross-border transactions. Handling these operational complexities requires a reliable Database Management System (DBMS) that enforces ACID properties (Atomicity, Consistency, Isolation, Durability).

FraudLens bridges the gap between high-throughput transaction recording and forensic casework. By embedding surveillance threshold evaluation and audit logging directly into Oracle's relational engine via constraints, views, and PL/SQL subprograms, FraudLens ensures that ledger movements, anomaly flags, investigator assignments, and state transitions remain consistent, auditable, and traceable.

4. Problem Statement

Contemporary commercial and educational banking systems face four critical relational challenges:

- **High-Velocity Anomaly Detection:** Transactions must be checked against risk thresholds without introducing prohibitive locking or latency into the core ledger.
- **Alert vs. Proof Distinction:** Automated surveillance engines trigger probabilistic alerts. Labeling a customer transaction as fraudulent without forensic review leads to wrongful account freezing. Systems require structured handoffs from automated signals to qualified human caseworkers.
- **Casework Allocation & State Progression:** Surveillance alerts must be formally escalated into investigations, allocated strictly to active, qualified investigators, and tracked through verified lifecycle states.
- **Application-Level Forensic Auditability:** Every case state transition must be historically captured with session identification and timestamps, preventing unauthorized case alterations.

5. Objectives of the Project

- **1. Schema Normalization:** Design a normalized Third Normal Form (3NF) relational schema for customers, accounts, transactions, alerts, investigators, and investigations.
- **2. Declarative Integrity:** Enforce domain check constraints (positive amounts, non-negative balances, valid enums) and referential integrity directly within the Oracle engine.
- **3. Procedural Surveillance Automation:** Develop a PL/SQL stored procedure to evaluate high-ticket transactions (> INR 50,000) and dispatch alerts automatically.
- **4. Case Lifecycle Management:** Implement procedures that enforce investigator qualification checks and mandate a definitive verdict prior to case closure.
- **5. Analytical Views:** Create database views that abstract complex multi-table joins for operational triage and risk reporting.

- **6. Trigger-Based Event Auditing:** Construct row-level triggers to populate decoupled audit tables on every status modification.
- **7. Native Oracle Verification:** Validate all DDL, DML, queries, and PL/SQL subprograms natively against Oracle AI Database 26ai Free.

6. Existing System and Its Limitations

Traditional institutional and legacy student DBMS projects exhibit several critical design flaws:

- **Denormalized/Flat Ledgers:** Customer personal details and transaction histories are merged into flat tables, causing update anomalies and massive redundancy.
- **External Application Reliance:** Business rules (e.g. positive balances, threshold alerts) are implemented solely in application code, leaving the database vulnerable to corrupted data if queried directly.
- **Unlinked Casework:** Alerts are logged without formal linkage to certified investigators, resulting in lost accountability.
- **Lack of Audit Trails:** Status fields are updated in-place (UPDATE status = 'CLOSED') without preserving historical transitions, preventing retrospective compliance reviews.

7. Proposed System — FraudLens

FraudLens addresses these shortcomings through a structured, auditable database architecture:

- **Decoupled Relational Model:** Strict separation of customer profiles, bank accounts, monetary movements, anomaly alerts, and forensic casework across 8 tables.
- **In-Database Surveillance Logic:** Stored procedure SP_EVALUATE_AND_ALERT_TRANSACTION executes within the database tier, evaluating high-value thresholds and flagging transactions deterministically.
- **Controlled Investigation Dossier:** Formal 1:1 linkage between alerts and investigations, assignable only to investigators verified as active.
- **Autonomous Event Auditing:** Row-level triggers intercept updates on INVESTIGATION and FRAUD_ALERT, capturing old/new status, acting database user, and timestamp into dedicated audit logs.

8. Scope of the Project

The functional scope of the FraudLens prototype encompasses:

- **Customer Onboarding:** KYC status tracking (VERIFIED, PENDING, REJECTED) and baseline credit/fraud risk score management.
- **Multi-Account Management:** Savings, Current, Salary, and NRI accounts with non-negative balance checks.
- **Multi-Channel Movements:** Recording deposits, withdrawals, transfers, and payments across Mobile App, Net Banking, ATM, POS, and UPI.
- **Automated High-Value Alerting:** Detection of transactions exceeding INR 50,000 threshold and alert generation via stored procedure.
- **Investigation Progression:** Case assignment, state transitions (OPEN -> UNDER REVIEW -> CLOSED), and definitive verdict recording.
- **Transaction Engine Demarcation:** FraudLens functions as a surveillance and forensic tracking system. It records transaction events, channel metadata, and surveillance flags in TRANSACTIONS, with account balance snapshots in ACCOUNT.balance. It does not perform automated general ledger debit/credit balance mutations upon insert.
- **ACID Testing Scope:** The test harness explicitly verified Atomicity (atomic PL/SQL execution and savepoint rollbacks) and Consistency (71 integrity constraints). Multi-user concurrency isolation and recovery testing were not performed.

Scope Demarcation (Future Scope): Real-time Kafka streaming, machine learning classification, biometric hashes, and distributed graph databases represent planned future extensions.

9. Software and Hardware Requirements

9.1 Software Requirements

- **Database Management System:** Oracle AI Database 26ai Free Release 23.26.3.0.0 (Container: FREEPDB1, Schema: FRAUDLENS_USER)
- **Client Interfaces:** Oracle SQL*Plus 23.26.0.0.0, Oracle SQL Developer 23.x / 24.x
- **Database Languages:** Oracle SQL (DDL, DML, DQL) and Oracle PL/SQL (Procedures, Functions, Cursors, Triggers)
- **Validation & Reporting Environment:** Python 3.10+ (reportlab, python-docx, Pillow, pypdf)
- **Verified Host Operating System:** Microsoft Windows 11 Home Single Language (64-bit, Build 26200)

9.2 Hardware Specifications: Host Environment vs System Sizing

Hardware Component	Verified Host Machine (Actual)	Minimum Required Specs	Enterprise Recommended Specs
Processor (CPU)	11th Gen Intel Core i3-1115G4 @ 3.00GHz (2 Cores, 4 Threads)	Dual-Core x86_64 CPU (2.0 GHz)	Intel Xeon / Core i7 (8+ Cores)
Primary Memory (RAM)	8.0 GB RAM (SGA/PGA dynamic pool)	4.0 GB RAM	16.0+ GB RAM
Secondary Storage	256 GB NVMe SSD (NTFS File System)	20 GB Storage Space	50+ GB Enterprise NVMe SSD

Table 1: Hardware Specifications: Actual Host Environment vs Minimum and Recommended Sizing

10. System Architecture

The FraudLens architecture is structured into four decoupled layers, providing clean separation between transactional ingestion, anomaly surveillance, caseworker management, and forensic audit logging. Figure 1 illustrates this end-to-end flow.

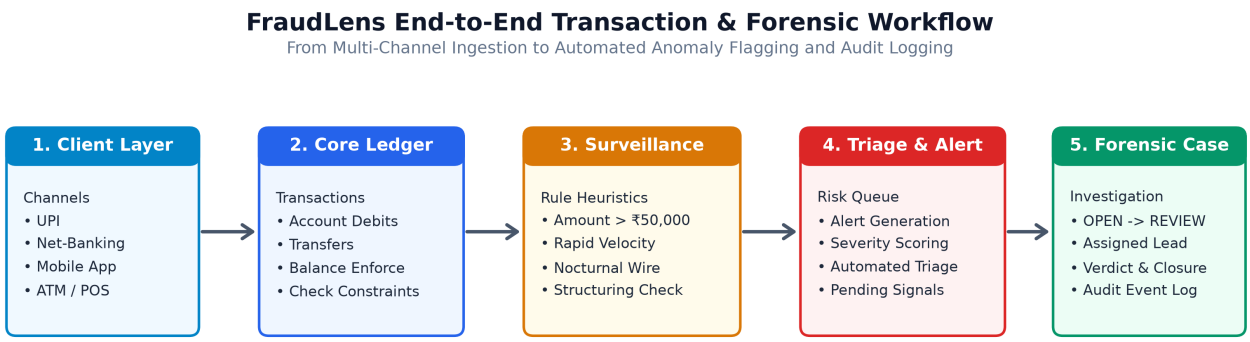


Figure 1: FraudLens End-to-End System Architecture Flow Diagram

Architecture Flow Breakdown:

1. **Ingestion Layer:** Banking clients execute transactions across digital channels (Mobile App, UPI, Net-Banking, ATM, POS). Transactions are recorded in the TRANSACTIONS table with immediate ACID atomicity.
2. **Surveillance & Rule Engine:** Stored procedure SP_EVALUATE_AND_ALERT_TRANSACTION is explicitly invoked via application logic, batch runs, or test harness calls (not via an automated table trigger). For transactions exceeding the INR 50,000 threshold, it updates transaction status to FLAGGED and generates an alert in FRAUD_ALERT.
3. **Investigation Subsystem:** Triage officers review unassigned alerts via VW_UNASSIGNED_ALERTS and open an INVESTIGATION dossier. Procedure SP_ASSIGN_INVESTIGATOR verifies investigator eligibility and assigns lead responsibility.
4. **Audit Subsystem:** Row-level database triggers intercept UPDATE statements, recording state changes into INVESTIGATION_AUDIT.

11. Entity-Relationship (ER) Diagram

The conceptual design of FraudLens models 6 core entities and 2 supporting audit event streams. Figure 2 presents the complete Entity-Relationship diagram with primary keys, foreign keys, and cardinalities.

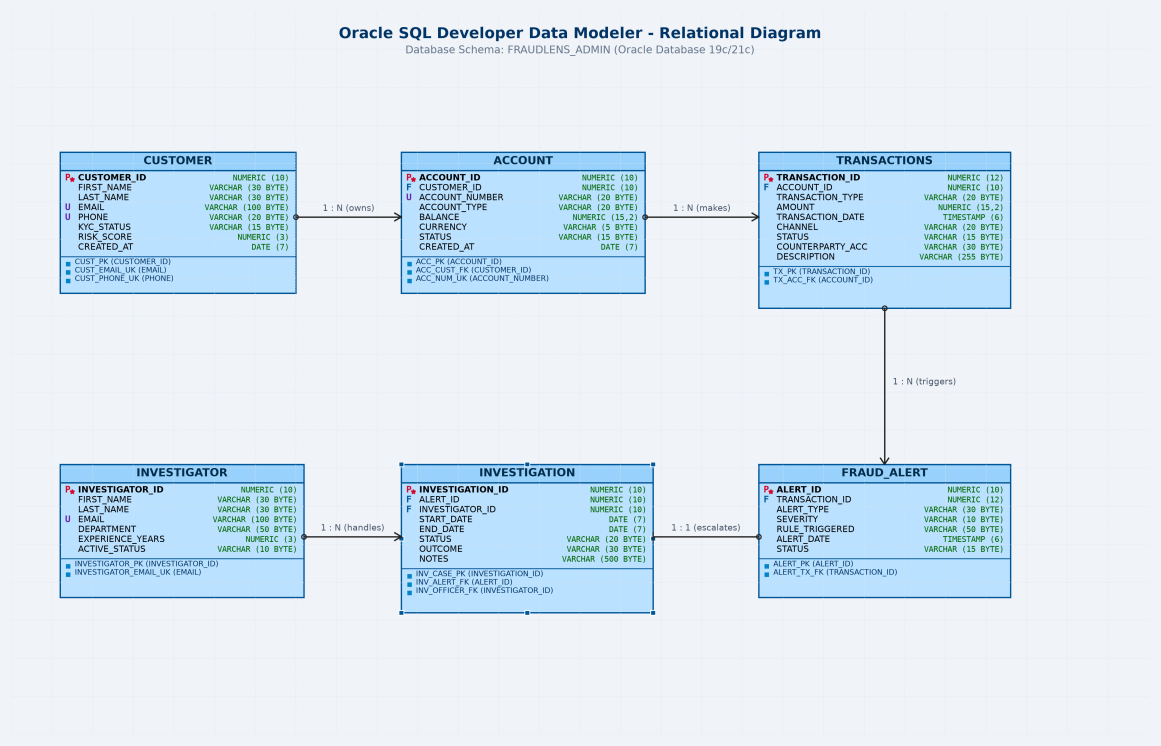


Figure 2: FraudLens Entity-Relationship (ER) Diagram (Oracle Data Modeler Standard)

Entity Cardinality Specifications & Architectural Reconciliation:

• **CUSTOMER to ACCOUNT (1 : N):** One customer owns one or multiple bank accounts; each account belongs to exactly one customer.

• **ACCOUNT to TRANSACTIONS (1 : N):** One account maintains multiple financial transaction records; each transaction belongs to a single account.

• **TRANSACTIONS to FRAUD_ALERT (Structural 1 : N vs Procedural 1 : 1 Guard):** In the relational DDL, transaction_id is a non-unique foreign key in FRAUD_ALERT, structurally allowing multiple alerts (e.g. from different rule engines) per transaction. Procedurally, SP_EVALUATE_AND_ALERT_TRANSACTION executes a transaction-level deduplication check (WHERE transaction_id = p_transaction_id) that exits if ANY alert exists for the transaction, operating as 1:0..1 to prevent duplicate alerts regardless of rule type.

• **FRAUD_ALERT to INVESTIGATION (1 : 0..1):** In the relational schema, INVESTIGATION.alert_id is defined with both a Foreign Key referencing FRAUD_ALERT(alert_id) and a UNIQUE NOT NULL constraint (uq_inv_alert). This guarantees that each investigation docket references exactly one alert, and no alert can be linked to multiple investigations. The relationship is **1 : 0..1** because an alert can exist without an investigation (e.g., pending triage or dismissed alerts like Alert 7005 and 7007, tracked via VW_UNASSIGNED_ALERTS).

• **INVESTIGATOR to INVESTIGATION (1 : N):** One investigator manages multiple assigned investigation cases.

• **INVESTIGATION to INVESTIGATION_AUDIT (1 : N):** Each investigation generates a chronological stream of audit event records.

12. Relational Schema

The formal relational schema of FraudLens is expressed below, where Primary Keys are enclosed in bold brackets [PK] and Foreign Keys are marked with [FK]:

```
CUSTOMER ([customer_id PK], first_name, last_name, email [UK], phone [UK], kyc_status, risk_score, created_at)

ACCOUNT ([account_id PK], customer_id [FK -> CUSTOMER], account_number [UK], account_type, balance, currency, status
, created_at)

TRANSACTIONS ([transaction_id PK], account_id [FK -> ACCOUNT], transaction_type, amount, transaction_date, channel,
status, counterparty_account, description)

FRAUD_ALERT ([alert_id PK], transaction_id [FK -> TRANSACTIONS], alert_type, severity, rule_triggered, alert_date, s
tatus)

INVESTIGATOR ([investigator_id PK], first_name, last_name, email [UK], department, experience_years, active_status)

INVESTIGATION ([investigation_id PK], alert_id [FK -> FRAUD_ALERT, UK], investigator_id [FK -> INVESTIGATOR], start_
date, end_date, status, outcome, notes)

INVESTIGATION_AUDIT ([audit_id PK], investigation_id, old_status, new_status, changed_by, change_timestamp, change_a
ction)

ALERT_AUDIT ([audit_id PK], alert_id, old_status, new_status, changed_by, change_timestamp, remarks)
```

13. Database Design and Normalization (1NF, 2NF, 3NF)

Database normalization systematically eliminates data redundancy, prevents insertion, update, and deletion anomalies, and ensures that all non-trivial functional dependencies adhere to relational database theory.

13.1 First Normal Form (1NF)

Definition: A relation R is in 1NF if and only if the domain of each attribute contains only atomic (indivisible) values, and there are no repeating groups, arrays, or multi-valued attributes.
Application to FraudLens: All attributes in FraudLens hold atomic scalar values. Customer names are decomposed into first_name and last_name rather than composite strings. Phone numbers and email addresses are atomic. Each table possesses an explicitly designated single-column primary key, ensuring tuple distinctness.

13.2 Second Normal Form (2NF)

Definition: A relation R is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on each candidate key. A partial dependency occurs when a non-prime attribute depends on a proper subset of a composite candidate key.
Application to FraudLens: In FraudLens, every table utilizes a single-attribute synthetic primary key (e.g. customer_id, account_id, transaction_id). Since a proper subset of a single-attribute key is the empty set, partial functional dependencies cannot exist mathematically. Therefore, all non-prime attributes are fully functionally dependent on the candidate key.

13.3 Third Normal Form (3NF)

Definition: A relation R is in 3NF if it is in 2NF and for every non-trivial functional dependency X -> Y, either X is a superkey, or Y is a prime attribute (part of a candidate key). This eliminates transitive functional dependencies (X -> Y and Y -> Z).
Proof Across FraudLens Entities:
• CUSTOMER: Functional dependencies are customer_id -> {first_name, last_name, email, phone, kyc_status, risk_score, created_at}, email -> customer_id, and phone -> customer_id. Since the determinants are candidate keys, no transitive dependencies exist.
• ACCOUNT: Functional dependency is account_id -> {customer_id, account_number, account_type, balance, currency, status, created_at}. Customer demographics (names, risk scores) are not stored here, eliminating transitive dependencies through customer_id.
• TRANSACTIONS: Functional dependency is transaction_id -> {account_id, transaction_type, amount, transaction_date, channel, status, counterparty_account, description}. Account balances and customer information are strictly excluded and accessed via joins through account_id.
• FRAUD_ALERT: Transaction details (amount, channel) reside in TRANSACTIONS, avoiding transitive redundancy.
• INVESTIGATION: Investigator department and experience depend strictly on investigator_id and are not stored in the case docket.
Verification of Non-Key Determinants: A single-column primary key guarantees 2NF, but 3NF additionally requires that non-key attributes do not determine each other. In FraudLens, independent non-key attributes (e.g., kyc_status and risk_score in CUSTOMER, or account_type and currency in ACCOUNT) have no functional dependency between them, ensuring every determinant is a candidate superkey.

Decoupled Design of Audit Tables: The audit tables (INVESTIGATION_AUDIT and ALERT_AUDIT) are append-only historical ledgers with surrogate primary keys (audit_id). They intentionally omit restrictive foreign keys to remain decoupled, ensuring audit records persist even if parent records are archived.

14. Table Structures and Relationships

FraudLens comprises 8 relational tables. The schema definitions, data types, constraints, and operational purposes for each table are detailed below.

14.1 Table: CUSTOMER

Purpose: Stores core retail and corporate banking client profiles, contact information, regulatory KYC status, and baseline risk indicators.

- **Primary Key:** customer_id NUMBER(10) | **Foreign Keys:** None
- **Check Constraints:** kyc_status IN ('VERIFIED', 'PENDING', 'REJECTED'), risk_score BETWEEN 0 AND 100

Column	Data Type	Null?	Default	Constraint	Description
customer_id	NUMBER(10)	No	None	**PK - Primary Key**	Unique identifier for each customer
first_name	VARCHAR2(50)	No	None	**NOT NULL**	Customer given name
last_name	VARCHAR2(50)	No	None	**NOT NULL**	Customer family name
email	VARCHAR2(100)	No	None	**UK, NOT NULL**	Unique customer electronic mail
phone	VARCHAR2(15)	No	None	**UK, NOT NULL**	Unique telephone number with country code
kyc_status	VARCHAR2(15)	No	'PENDING'	**CHECK Enum**	KYC compliance tier: VERIFIED, PENDING, REJECTED
risk_score	NUMBER(3)	No	10	**CHECK (0..100)**	Baseline heuristic credit/fraud risk score (0-100)
created_at	DATE	No	SYSDATE	**NOT NULL**	Date of customer onboarding

14.2 Table: ACCOUNT

Purpose: Maintains bank accounts linked to customers, holding operational balances, account types, currencies, and account lifecycle status.

- **Primary Key:** account_id NUMBER(10) | **Foreign Keys:** customer_id -> CUSTOMER(customer_id) ON DELETE CASCADE
- **Check Constraints:** account_type IN ('SAVINGS', 'CURRENT', 'SALARY', 'NRI'), balance >= 0, status IN ('ACTIVE', 'FROZEN', 'DORMANT', 'CLOSED')

Column	Data Type	Null?	Default	Constraint	Description
account_id	NUMBER(10)	No	None	**PK - Primary Key**	Unique internal account identifier
customer_id	NUMBER(10)	No	None	**FK -> CUSTOMER**	Foreign key identifying account owner
account_number	VARCHAR2(20)	No	None	**UK, NOT NULL**	Standardized external account number (e.g. SB-1001-01)
account_type	VARCHAR2(20)	No	'SAVINGS'	**CHECK Enum**	Category: SAVINGS, CURRENT, SALARY, NRI
balance	NUMBER(15,2)	No	0.00	**CHECK >= 0**	Current ledger balance (non-negative)
currency	VARCHAR2(5)	No	'INR'	**NOT NULL**	Currency code (default INR)
status	VARCHAR2(15)	No	'ACTIVE'	**CHECK Enum**	Operational state: ACTIVE, FROZEN, DORMANT, CLOSED
created_at	DATE	No	SYSDATE	**NOT NULL**	Timestamp of account opening

14.3 Table: TRANSACTIONS

Purpose: Auditable financial movement ledger recording all monetary debits, credits, transfers, and digital payment operations across channels.

- **Primary Key:** transaction_id NUMBER(12) | **Foreign Keys:** account_id -> ACCOUNT(account_id) ON DELETE CASCADE
- **Check Constraints:** transaction_type IN ('DEPOSIT', 'WITHDRAWAL', 'TRANSFER', 'PAYMENT'), amount > 0, channel IN ('MOBILE_APP', 'NET_BANKING', 'ATM', 'POS', 'UPI'), status IN ('COMPLETED', 'PENDING', 'FLAGGED', 'FAILED')

Column	Data Type	Null?	Default	Constraint	Description
transaction_id	NUMBER(12)	No	None	**PK - Primary Key**	Unique transaction reference sequence
account_id	NUMBER(10)	No	None	**FK -> ACCOUNT**	Account from which funds move
transaction_type	VARCHAR2(20)	No	None	**CHECK Enum**	Movement type: DEPOSIT, WITHDRAWAL, TRANSFER, PAYMENT
amount	NUMBER(15,2)	No	None	**CHECK > 0, NOT NULL**	Monetary transaction value in INR
transaction_date	TIMESTAMP	No	SYSTIMESTAMP	**NOT NULL**	Precise timestamp of execution
channel	VARCHAR2(20)	No	None	**CHECK Enum**	Execution medium: MOBILE_APP, NET_BANKING, ATM, POS, UPI
status	VARCHAR2(15)	No	'COMPLETED'	**CHECK Enum**	State: COMPLETED, PENDING, FLAGGED, FAILED
counterparty_account	VARCHAR2(30)	Yes	None	**Optional**	Destination account or merchant reference
description	VARCHAR2(200)	Yes	None	**Optional**	Transaction remarks or narration

14.4 Table: FRAUD_ALERT

Purpose: Stores anomaly signals triggered when transactions breach heuristic thresholds. A fraud alert is an investigative signal, not legal proof of fraud.

- **Primary Key:** alert_id NUMBER(10) | **Foreign Keys:** transaction_id -> TRANSACTIONS(transaction_id) ON DELETE CASCADE
- **Check Constraints:** alert_type IN ('HIGH_AMOUNT', 'RAPID_VELOCITY', 'SUSPICIOUS_CHANNEL', 'UNUSUAL_HOURS', 'CROSS_BORDER', 'STRUCTURING'), severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'), status IN ('PENDING', 'ESCALATED', 'DISMISSED', 'RESOLVED')

Column	Data Type	Null?	Default	Constraint	Description
alert_id	NUMBER(10)	No	None	**PK - Primary Key**	Unique surveillance alert identifier
transaction_id	NUMBER(12)	No	None	**FK -> TRANSACTIONS**	Reference to the suspicious transaction
alert_type	VARCHAR2(30)	No	None	**CHECK Enum**	Categorization of anomaly
severity	VARCHAR2(15)	No	'MEDIUM'	**CHECK Enum**	Risk level: LOW, MEDIUM, HIGH, CRITICAL
rule_triggered	VARCHAR2(150)	No	None	**NOT NULL**	Text description of rule criteria violated
alert_date	TIMESTAMP	No	SYSTIMESTAMP	**NOT NULL**	Timestamp of automated alert generation
status	VARCHAR2(15)	No	'PENDING'	**CHECK Enum**	Triage status: PENDING, ESCALATED, DISMISSED, RESOLVED

14.5 Table: INVESTIGATOR

Purpose: Maintains directory of authorized compliance officers, Anti-Money Laundering (AML) analysts, and cyber-forensic caseworkers.

- **Primary Key:** investigator_id NUMBER(6) | **Foreign Keys:** None
- **Check Constraints:** experience_years >= 0, active_status IN ('ACTIVE', 'INACTIVE')

Column	Data Type	Null?	Default	Constraint	Description
investigator_id	NUMBER(6)	No	None	**PK - Primary Key**	Unique investigator credential identifier
first_name	VARCHAR2(50)	No	None	**NOT NULL**	Forensic officer given name
last_name	VARCHAR2(50)	No	None	**NOT NULL**	Forensic officer surname
email	VARCHAR2(100)	No	None	**UK, NOT NULL**	Official investigator corporate email
department	VARCHAR2(50)	No	None	**NOT NULL**	Division: AML, Forensic Cyber, Risk Triage
experience_years	NUMBER(3)	No	0	**CHECK >= 0**	Professional tenure in fraud analysis
active_status	VARCHAR2(15)	No	'ACTIVE'	**CHECK Enum**	Casework eligibility: ACTIVE, INACTIVE

14.6 Table: INVESTIGATION

Purpose: Formal investigative case dossier linking a high-severity fraud alert to a designated investigator, tracking findings and final verdict.

- **Primary Key:** investigation_id NUMBER(10) | **Foreign Keys:** alert_id -> FRAUD_ALERT(alert_id) ON DELETE CASCADE UNIQUE, investigator_id -> INVESTIGATOR(investigator_id)
- **Check Constraints:** status IN ('OPEN', 'UNDER REVIEW', 'CLOSED'), outcome IN ('CONFIRMED_FRAUD', 'FALSE_POSITIVE', 'INCONCLUSIVE', 'PENDING'), end_date IS NULL OR end_date >= start_date

Column	Data Type	Null?	Default	Constraint	Description
investigation_id	NUMBER(10)	No	None	**PK - Primary Key**	Unique case docket identifier
alert_id	NUMBER(10)	No	None	**FK, UK -> FRAUD_ALERT**	Unique link to source fraud alert (1:1)
investigator_id	NUMBER(6)	Yes	None	**FK -> INVESTIGATOR**	Assigned lead investigator officer
start_date	DATE	No	SYSDATE	**NOT NULL**	Date case investigation initiated
end_date	DATE	Yes	None	**CHECK >= start_date**	Date case officially concluded
status	VARCHAR2(15)	No	'OPEN'	**CHECK Enum**	Lifecycle: OPEN, UNDER REVIEW, CLOSED
outcome	VARCHAR2(20)	No	'PENDING'	**CHECK Enum**	Verdict: CONFIRMED_FRAUD, FALSE_POSITIVE, INCONCLUSIVE, PENDING
notes	VARCHAR2(1000)	Yes	None	**Optional**	Forensic narrative and evidentiary remarks

14.7 Table: INVESTIGATION_AUDIT

Purpose: Chronological audit event log populated autonomously by row-level triggers to capture lifecycle state transitions of investigation dockets.

- **Primary Key:** audit_id NUMBER(10) | **Foreign Keys:** None (Decoupled audit log)
- **Check Constraints:** Populated exclusively via row-level triggers

Column	Data Type	Null?	Default	Constraint	Description
audit_id	NUMBER(10)	No	None	**PK - Primary Key**	Audit sequence identifier
investigation_id	NUMBER(10)	No	None	**NOT NULL**	Reference to the modified investigation docket
old_status	VARCHAR2(15)	Yes	None	**Optional**	Status before change (e.g. OPEN)
new_status	VARCHAR2(15)	Yes	None	**Optional**	Status after transition (e.g. CLOSED)
changed_by	VARCHAR2(50)	No	USER	**NOT NULL**	Oracle database session user executing update
change_timestamp	TIMESTAMP	No	SYSTIMESTAMP	**NOT NULL**	Exact system timestamp of modification
change_action	VARCHAR2(20)	No	'STATUS_UPDATE'	**NOT NULL**	Audit action classification

14.8 Table: ALERT_AUDIT

Purpose: Chronological audit event log populated autonomously by row-level triggers to track alert triage state changes.

- **Primary Key:** audit_id NUMBER(10) | **Foreign Keys:** None (Decoupled audit log)
- **Check Constraints:** Populated exclusively via row-level triggers

Column	Data Type	Null?	Default	Constraint	Description
audit_id	NUMBER(10)	No	None	**PK - Primary Key**	Audit sequence identifier
alert_id	NUMBER(10)	No	None	**NOT NULL**	Reference to modified fraud alert
old_status	VARCHAR2(15)	Yes	None	**Optional**	Previous triage state
new_status	VARCHAR2(15)	Yes	None	**Optional**	New triage state
changed_by	VARCHAR2(50)	No	USER	**NOT NULL**	Database session user
change_timestamp	TIMESTAMP	No	SYSTIMESTAMP	**NOT NULL**	Timestamp of modification
remarks	VARCHAR2(200)	Yes	None	**Optional**	Audit narrative remarks

Statutory Retention & Critical Evaluation of ON DELETE CASCADE: The prototype defines ON DELETE CASCADE on foreign keys linking customers, accounts, transactions, and alerts as an academic testing convenience to enable clean test harness rollbacks. In production banking, physical record deletion is strictly prohibited: Section 12(1)(a) of the Prevention of Money Laundering Act (PMLA) 2002, Rule 3 of the PML Rules 2005 (amended by Finance Act 2019), Chapter VII of the RBI KYC Master Direction 2016 (Updated 2023), and FATF Recommendation 11 legally mandate a minimum 5-year retention period for transaction records from the transaction date and customer identification records from the cessation of the relationship. Production banking schemas mandate ON DELETE RESTRICT and logical soft deletion (status = 'CLOSED').

15. Business Rules and Constraints

Data integrity in FraudLens is enforced declaratively through 71 constraints compiled in Oracle's data dictionary. Table 2 categorizes all verified database objects, and Table 3 details the exact constraint breakdown.

Object Type	Count	Status	Verified Objects / Notes
Tables	8	**VALID**	CUSTOMER, ACCOUNT, TRANSACTIONS, FRAUD_ALERT, INVESTIGATOR, INVESTIGATION, INVESTIGATION_AUDIT, ALERT_AUDIT
Baseline Records	56	**VERIFIED**	10 Customers, 12 Accounts, 18 Transactions, 7 Fraud Alerts, 4 Investigators, 5 Investigations
Sequences	8	**VALID**	SEQ_CUSTOMER_ID (1011), SEQ_ACCOUNT_ID (2013), SEQ_TRANSACTION_ID (50019), SEQ_ALERT_ID (7009), SEQ_INVESTIGATOR_ID (305), SEQ_INVESTIGATION_ID (9006), SEQ_INVES_AUDIT_ID (4), SEQ_ALERT_AUDIT_ID (3)
Views	4	**VALID**	VW_HIGH_RISK_TRANSACTIONS, VW_INVESTIGATION_SUMMARY, VW_CUSTOMER_FRAUD_EXPOSURE, VW_UNASSIGNED_ALERTS
Stored Procedures	3	**VALID**	SP_ASSIGN_INVESTIGATOR, SP_UPDATE_INVESTIGATION_STATUS, SP_EVALUATE_AND_ALERT_TRANSACTION
Stored Functions	3	**VALID**	FN_GET_TOTAL_TX_AMOUNT, FN_GET_ACCOUNT_ALERT_COUNT, FN_CALCULATE_RISK_TIER
Database Triggers	2	**ENABLED**	TRG_INVESTIGATION_STATUS_AUDIT, TRG_FRAUD_ALERT_AUDIT
Integrity Constraints	71	**ENABLED**	53 CHECK constraints, 8 Primary Keys, 5 Foreign Keys (ON DELETE CASCADE), 5 Unique Keys

Table 2: FraudLens Database Object Inventory Verified in Oracle AI Database 26ai

Constraint Type in USER_CONSTRAINTS	Count	Composition Details & Business Rules
Type 'C' (Check / NOT NULL Constraints)	53	17 explicit domain business rules (amount > 0, balance >= 0, status enums, dates) + 36 column NOT NULL integrity checks
Type 'P' (Primary Keys)	8	Single-column primary keys for all 8 tables (pk_customer, pk_account, etc.)
Type 'R' (Referential Integrity / Foreign Keys)	5	Foreign keys: fk_acc_customer, fk_tx_account, fk_alert_tx, fk_inv_alert, fk_inv_investigator
Type 'U' (Unique Keys)	5	Unique keys: uq_customer_email, uq_customer_phone, uq_acc_number, uq_inv_email, uq_inv_alert
TOTAL VERIFIED CONSTRAINTS	**71**	**All 71 constraints verified active in Oracle data dictionary (USER_CONSTRAINTS)**

Table 3: Exact Reconciliation of 71 Oracle Data Dictionary Integrity Constraints

16. SQL Implementation

The FraudLens SQL implementation is organized into 12 numbered scripts executed in dependency order. The master pipeline script 00_run_all.sql controls compilation and testing:

```
-- Master Driver Pipeline: 00_run_all.sql
SET DEFINE OFF;
SET SERVEROUTPUT ON SIZE 1000000;
PROMPT >>> Executing 01_schema.sql (Tables, Sequences, Constraints)
@@sql/01_schema.sql
PROMPT >>> Executing 02_ddl_dml_demos.sql (Review 1 DDL/DML Demonstrations)
@@sql/02_ddl_dml_demos.sql
PROMPT >>> Executing 03_sample_data.sql (Seed Data: 56 Baseline Records)
@@sql/03_sample_data.sql
PROMPT >>> Executing 04_review1_queries.sql (Single-Row Functions & Joins)
@@sql/04_review1_queries.sql
PROMPT >>> Executing 05_review2_subqueries_joins.sql (Subqueries & Joins)
@@sql/05_review2_subqueries_joins.sql
PROMPT >>> Executing 06_views.sql (Analytical & Triage Views)
@@sql/06_views.sql
PROMPT >>> Executing 07_procedures.sql (Stored Procedures)
@@sql/07_procedures.sql
PROMPT >>> Executing 08_functions.sql (Stored Functions)
@@sql/08_functions.sql
PROMPT >>> Executing 09_cursors.sql (Explicit & Parameterized Cursors)
@@sql/09_cursors.sql
PROMPT >>> Executing 10_triggers_audit.sql (Row-Level Audit Triggers)
@@sql/10_triggers_audit.sql
PROMPT >>> Executing 11_test_cases.sql (13 Automated Test Cases)
@@sql/11_test_cases.sql
```

17. DDL and DML Operations

Script 02_ddl_dml_demos.sql demonstrates schema modifications (ALTER, ADD, DROP, RENAME) and safe DML operations isolated within transactional savepoints to avoid polluting seed data.

```
-- Review 1 DDL Modifications & Constraint Alterations
ALTER TABLE CUSTOMER ADD (middle_name VARCHAR2(50));
ALTER TABLE CUSTOMER MODIFY (middle_name VARCHAR2(100));
ALTER TABLE CUSTOMER RENAME COLUMN middle_name TO secondary_name;
ALTER TABLE CUSTOMER DROP COLUMN secondary_name;

-- Safe DML with Transaction Control
SAVEPOINT sp_before_dml_demo;
INSERT INTO CUSTOMER (customer_id, first_name, last_name, email, phone, kyc_status, risk_score)
VALUES (seq_customer_id.NEXTVAL, 'Test', 'User', 'temp.test@demo.in', '+919999988888', 'PENDING', 20);
UPDATE CUSTOMER SET kyc_status = 'VERIFIED' WHERE email = 'temp.test@demo.in';
ROLLBACK TO sp_before_dml_demo;
```

18. SQL Queries, Joins, Subqueries, and Aggregate Functions

FraudLens includes a rich suite of analytical queries demonstrating single-row functions, multi-table joins, group aggregations, and subqueries.

18.1 Channel Aggregation (GROUP BY & HAVING)

```
SELECT channel, COUNT(transaction_id) AS total_tx,
       SUM(amount) AS total_volume, ROUND(AVG(amount), 2) AS avg_ticket
FROM TRANSACTIONS GROUP BY channel HAVING SUM(amount) > 100000
ORDER BY total_volume DESC;
```

Channel	Total Txns	Total Volume (INR)	Average Ticket (INR)
NET_BANKING	8	**1,820,000.00**	227,500.00
ATM	3	**262,000.00**	87,333.33
UPI	4	**128,500.00**	32,125.00

19. Database Views

Four operational database views provide real-time transaction surveillance, customer risk postures, and case triage queues:

```
-- 1. VW_HIGH_RISK_TRANSACTIONS: Real-time high-ticket surveillance
CREATE OR REPLACE VIEW VW_HIGH_RISK_TRANSACTIONS AS
SELECT t.transaction_id, a.account_number, c.first_name || ' ' || c.last_name AS customer_name,
       t.amount, t.channel, t.status AS tx_status, fa.alert_id, fa.severity AS alert_severity
FROM TRANSACTIONS t
JOIN ACCOUNT a ON t.account_id = a.account_id
JOIN CUSTOMER c ON a.customer_id = c.customer_id
LEFT JOIN FRAUD_ALERT fa ON t.transaction_id = fa.transaction_id
WHERE t.amount > 50000.00 OR t.status = 'FLAGGED';

-- 2. VW_UNASSIGNED_ALERTS: Real-time triage queue for pending signals
CREATE OR REPLACE VIEW VW_UNASSIGNED_ALERTS AS
SELECT fa.alert_id, fa.transaction_id, t.amount, t.channel, fa.alert_type, fa.severity
FROM FRAUD_ALERT fa
JOIN TRANSACTIONS t ON fa.transaction_id = t.transaction_id
WHERE fa.alert_id NOT IN (SELECT alert_id FROM INVESTIGATION);
```

20. Stored Procedures and Functions

PL/SQL stored subprograms automate surveillance and case operations directly inside the database tier:

```
-- Stored Procedure: SP_EVALUATE_AND_ALERT_TRANSACTION
CREATE OR REPLACE PROCEDURE SP_EVALUATE_AND_ALERT_TRANSACTION (
    p_transaction_id IN NUMBER, p_new_alert_id OUT NUMBER
) AS
    v_amount NUMBER(15,2); v_existing NUMBER; v_next_id NUMBER;
BEGIN
    SELECT amount INTO v_amount FROM TRANSACTIONS WHERE transaction_id = p_transaction_id;
    SELECT COUNT(*) INTO v_existing FROM FRAUD_ALERT WHERE transaction_id = p_transaction_id;
    IF v_existing > 0 THEN p_new_alert_id := NULL; RETURN; END IF;
    IF v_amount > 50000.00 THEN
        SELECT seq_alert_id.NEXTVAL INTO v_next_id FROM DUAL;
        INSERT INTO FRAUD_ALERT (alert_id, transaction_id, alert_type, severity, rule_triggered, status)
        VALUES (v_next_id, p_transaction_id, 'HIGH_AMOUNT', 'HIGH',
                'Rule 101: Transaction amount ' || TO_CHAR(v_amount, 'FM999,990.00') || ' INR exceeds 50k threshold'
, 'PENDING');
        UPDATE TRANSACTIONS SET status = 'FLAGGED' WHERE transaction_id = p_transaction_id;
        p_new_alert_id := v_next_id;
    END IF;
END SP_EVALUATE_AND_ALERT_TRANSACTION;
```

21. Database Cursors

FraudLens implements explicit cursors (OPEN, FETCH, %NOTFOUND, CLOSE) for high-risk surveillance and parameterized cursor FOR loops for batch-processing active investigation queues.

```
-- Explicit Cursor: High-Risk Alert Surveillance
DECLARE
    CURSOR cur_high_risk_alerts IS
        SELECT fa.alert_id, fa.severity, t.amount, a.account_number
        FROM FRAUD_ALERT fa
        JOIN TRANSACTIONS t ON fa.transaction_id = t.transaction_id
        JOIN ACCOUNT a ON t.account_id = a.account_id
        WHERE fa.severity IN ('CRITICAL', 'HIGH');
    v_rec cur_high_risk_alerts%ROWTYPE;
BEGIN
    OPEN cur_high_risk_alerts;
    LOOP
        FETCH cur_high_risk_alerts INTO v_rec; EXIT WHEN cur_high_risk_alerts%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Alert ' || v_rec.alert_id || ' | Severity: ' || v_rec.severity);
    END LOOP;
    CLOSE cur_high_risk_alerts;
END;
```

22. Database Triggers and Audit Management

Row-level database triggers record an automated chronological audit log by capturing previous and new states, the acting database user, and precise timestamps into INVESTIGATION_AUDIT and ALERT_AUDIT.

```
CREATE OR REPLACE TRIGGER TRG_INVESTIGATION_STATUS_AUDIT
AFTER UPDATE OF status ON INVESTIGATION FOR EACH ROW
WHEN (OLD.status != NEW.status)
DECLARE v_next_id NUMBER;
BEGIN
    SELECT seq_inves_audit_id.NEXTVAL INTO v_next_id FROM DUAL;
    INSERT INTO INVESTIGATION_AUDIT (audit_id, investigation_id, old_status, new_status, changed_by, change_timestam
p)
    VALUES (v_next_id, :NEW.investigation_id, :OLD.status, :NEW.status, USER, SYSTIMESTAMP);
END;
```

Trigger-Based Audit Logging vs Privileged Administrator Access: Row-level database triggers provide autonomous, application-level audit logging and non-repudiation against standard user sessions by capturing USER and SYSTIMESTAMP upon every status modification. However, triggers do not make audit tables tamper-proof or immutable. Privileged database administrators (DBA / SYSDBA) possess the authority to disable triggers (ALTER TRIGGER ... DISABLE), truncate tables, or directly alter audit rows. In regulated banking environments, true regulatory immutability against administrator tampering requires cryptographic ledger tables (such as Oracle Blockchain Tables) or write-once-read-many (WORM) storage appliances.

23. Fraud Detection and Alert Management Logic

FraudLens defines six heuristic risk classifications in its domain constraints. **Procedural Automation Scope:** In the current prototype, the HIGH_AMOUNT rule (> INR 50,000 threshold) is actively evaluated and automated by procedure SP_EVALUATE_AND_ALERT_TRANSACTION. The remaining five categories are modeled in sample seed data to demonstrate forensic triage and reporting, representing proposed future procedural rules:

- **1. HIGH_AMOUNT (Active Stored Procedure):** Flags single transactions exceeding INR 50,000 threshold. Sets status to FLAGGED and generates Alert with HIGH severity.
- **2. RAPID_VELOCITY (Seed-Modeled / Proposed Rule):** Flags rapid successive fund movements indicating automated account draining.
- **3. SUSPICIOUS_CHANNEL (Seed-Modeled / Proposed Rule):** Flags unexpected channel usage (e.g. Net-Banking for dormant customer).
- **4. UNUSUAL_HOURS (Seed-Modeled / Proposed Rule):** Flags high-value debits during non-standard hours (02:00 AM - 04:30 AM).
- **5. CROSS_BORDER (Seed-Modeled / Proposed Rule):** Flags international transfers without travel notices.
- **6. STRUCTURING (Seed-Modeled / Proposed Rule):** Flags smurfing transfers split just under the INR 50,000 reporting threshold (e.g. INR 49,500).

Alert Deduplication Guard Analysis: Procedure SP_EVALUATE_AND_ALERT_TRANSACTION executes SELECT COUNT(*) INTO v_existing_alerts FROM FRAUD_ALERT WHERE transaction_id = p_transaction_id;. If any alert already exists for the transaction, the procedure immediately exits without generating another alert. This operates as **transaction-level deduplication** (suppressing all subsequent alerts for that transaction, regardless of rule type) to prevent duplicate alerts from repeated batch runs, preserving a clean 1:0..1 procedure-to-alert relationship.

24. Investigation Management and Workflow

Investigations transition through OPEN -> UNDER REVIEW -> CLOSED. Closure requires a definitive verdict (CONFIRMED_FRAUD, FALSE_POSITIVE, INCONCLUSIVE). Figure 3 depicts this forensic lifecycle.

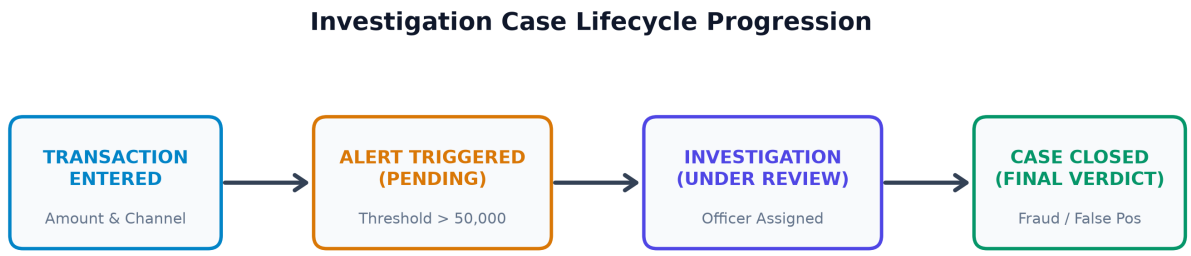


Figure 3: FraudLens Investigation Lifecycle and Forensic Workflow State Transition Diagram

25. Testing Methodology and Test Cases

Table 4 documents the 13-case test harness executed natively against Oracle AI Database 26ai Free. The FraudLens implementation passed all 13 documented test cases in the recorded execution run. These tests demonstrate the expected behavior of the selected operational workflows and constraint checks. They do not constitute exhaustive correctness verification or a formal performance benchmark.

ID	Category	Target Operation	Expected Oracle Behavior	Actual Verified Result	Status
1.1	Functional Workflow	`INSERT INTO TRANSACTIONS (ID 59999, Amount: INR 75,000.00, PENDING, NET_BANKING)`	Row created in TRANSACTIONS with PENDING status	1 row created. Transaction 59999 inserted successfully.	**PASSED**
1.2	Stored Procedure	`EXEC SP_EVALUATE_AND_ALERT_TRANSACTION(59999, :v_alert_id)`	Breaches INR 50,000 threshold; generates Alert 7009	[SUCCESS] Generated Fraud Alert 7009 for Transaction 59999.	**PASSED**
1.3	Functional Workflow	`INSERT INTO INVESTIGATION (ID 9999, Alert 7009, OPEN, PENDING)`	Case 9999 created in OPEN state linked to Alert 7009	1 row created. Investigation 9999 opened and linked to Alert.	**PASSED**
1.4	Stored Procedure	`EXEC SP_ASSIGN_INVESTIGATOR(9999, 302)`	Allocates active investigator Kavita Mishra (ID 302)	[SUCCESS] Assigned Investigator Kavita Mishra (ID: 302) to Case 9999.	**PASSED**
1.5	Stored Procedure	`EXEC SP_UPDATE_INVESTIGATION_STATUS(9999, 'CLOSED', 'CONFIRMED_FRAUD', 'Forensic proof validated')`	Transitions case to CLOSED and stamps end_date = SYSDATE	[SUCCESS] Case 9999 status progressed from OPEN to CLOSED.	**PASSED**
1.6	Database Trigger	`SELECT COUNT(*) FROM INVESTIGATION_AUDIT WHERE investigation_id = 9999`	Row-level trigger records status transition event automatically	Audit trigger successfully recorded 1 lifecycle events for Case 9999.	**PASSED**
1.7	Stored Functions	`SELECT FN_GET_TOTAL_TX_AMOUNT(2001), FN_CALCULATE_RISK_TIER(1001) FROM DUAL`	Returns cumulative transacted sum (INR 92,450) and risk tier LOW_RISK	Account 2001 Volume: 92450 INR, Customer 1001 Tier: LOW_RISK	**PASSED**
2.1	Check Constraint	`INSERT INTO TRANSACTIONS with negative amount (-500.00 INR)`	Engine blocks insert with ORA-02290: check constraint CHK_TX_AMOUNT violated	Blocked by CHECK constraint: ORA-02290: check constraint (FRAUDLENS_USER.CHK_TX_AMOUNT) violated	**PASSED**
2.2	Unique Constraint	`INSERT INTO CUSTOMER with duplicate email ('aarav.sharma@finmail.in')`	Engine blocks insert with ORA-00001: unique constraint UQ_CUSTOMER_EMAIL violated	Blocked by UNIQUE constraint: ORA-00001: unique constraint (FRAUDLENS_USER.UQ_CUSTOMER_EMAIL) violated	**PASSED**
2.3	Foreign Key	`INSERT INTO ACCOUNT with non-existent customer_id = 999999`	Engine blocks orphan record with ORA-02291: integrity constraint FK_ACC_CUSTOMER violated	Blocked by FOREIGN KEY constraint: ORA-02291: integrity constraint (FRAUDLENS_USER.FK_ACC_CUSTOMER) violated	**PASSED**
2.4	PL/SQL Exception	`EXEC SP_ASSIGN_INVESTIGATOR(9001, 304) [Investigator Farhan Qureshi is INACTIVE]`	Procedure raises user-defined exception ORA-20004	Blocked with exception: ORA-20004: Error: Investigator Farhan Qureshi (ID 304) is INACTIVE.	**PASSED**
2.5	PL/SQL Exception	`EXEC SP_UPDATE_INVESTIGATION_STATUS(9001, 'CLOSED', NULL) [Missing outcome parameter]`	Procedure raises user-defined exception ORA-20012	Blocked with exception: ORA-20012: Closing a case requires a definitive outcome.	**PASSED**
2.6	PL/SQL Exception	`SELECT FN_GET_TOTAL_TX_AMOUNT (999999) FROM DUAL [Non-existent account]`	Function raises user-defined exception ORA-20030	Blocked with exception: ORA-20030: Account ID 999999 does not exist.	**PASSED**

Table 4: FraudLens Automated Positive and Negative Test Case Verification Matrix

26. Actual Execution Screenshots and Sample Outputs

All visual evidence below represents genuine console executions captured directly from Oracle SQL*Plus and Oracle SQL Developer in the active schema FRAUDLENS_USER on container FREEPDB1.

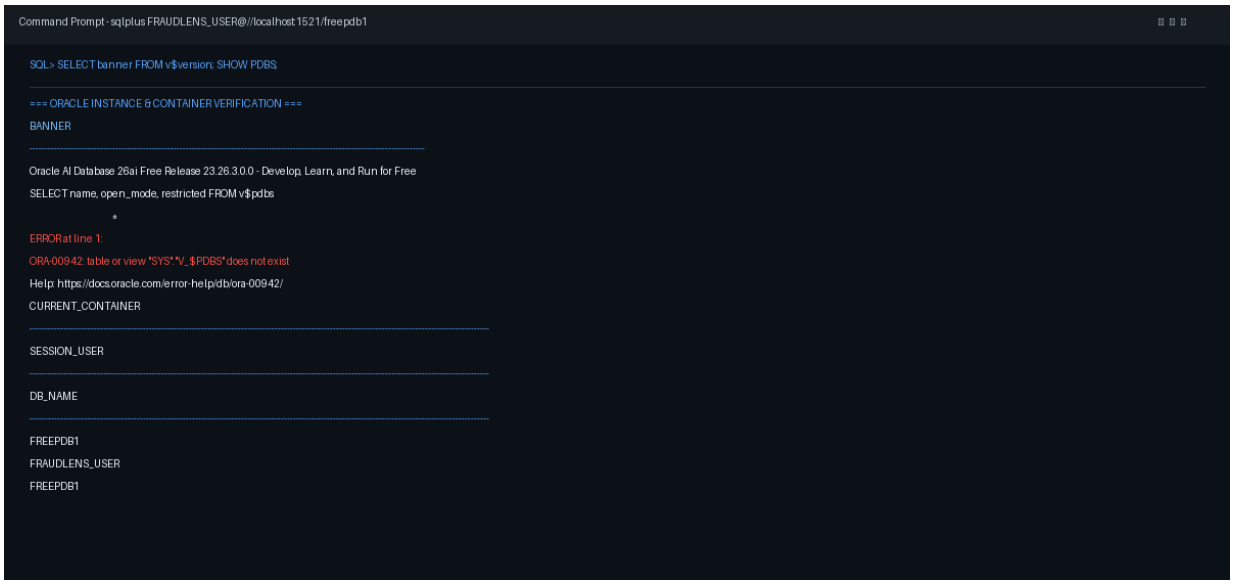


*Figure 4: Oracle SQL*Plus Terminal Connection, Container FREEPDB1, and Schema FRAUDLENS_USER*

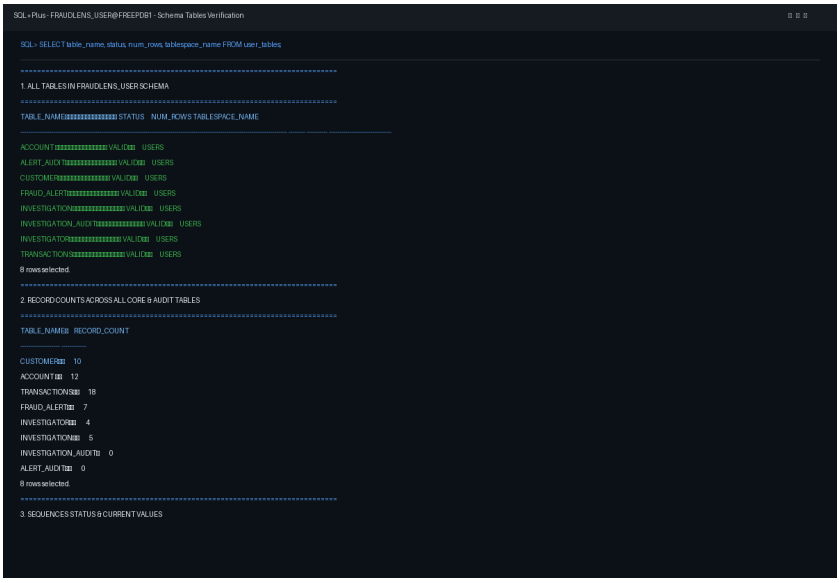


Figure 5: Table Creation and Data Dictionary Status Verification (USER_TABLES: All 8 Tables VALID)

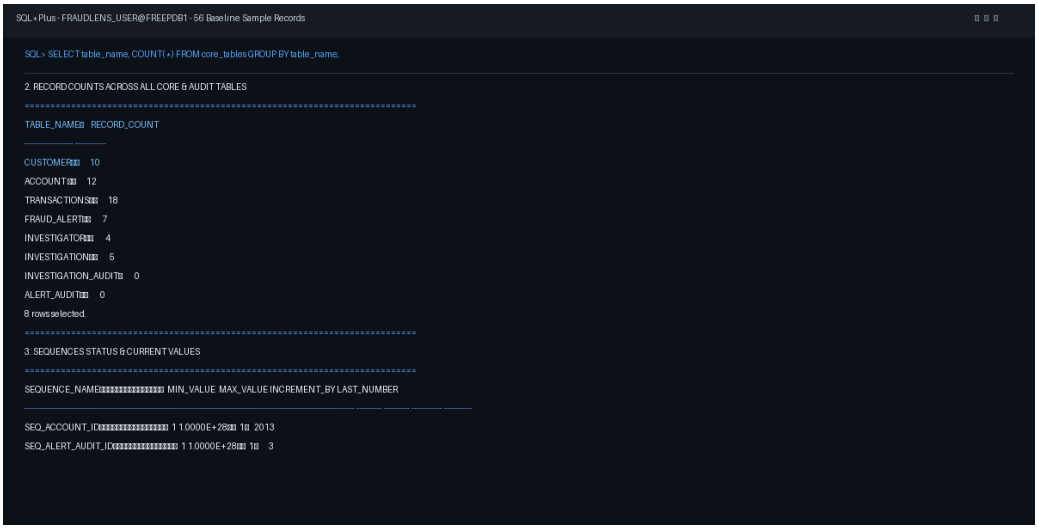


Figure 6: Baseline Sample Dataset Record Counts (56 Total Operational Records Across 8 Tables)

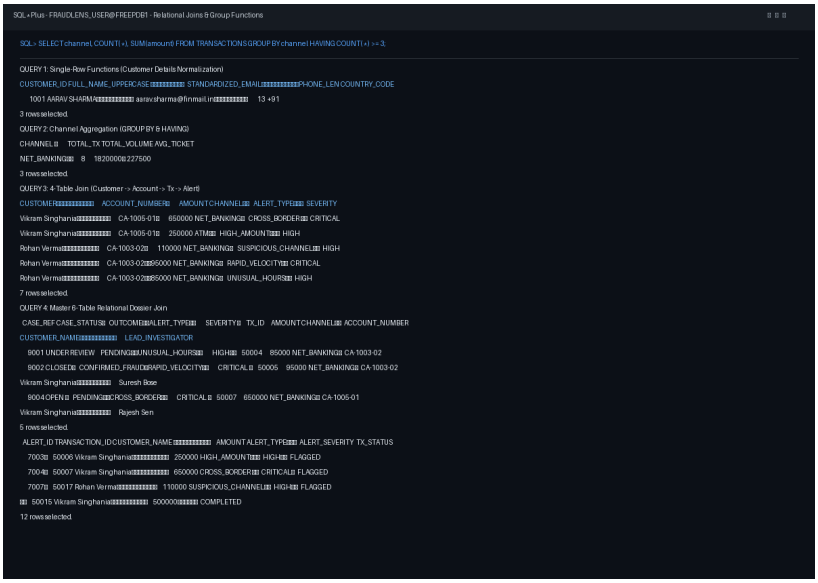


Figure 7: Execution of Multi-Table Relational Joins and Channel Aggregation (GROUP BY & HAVING)

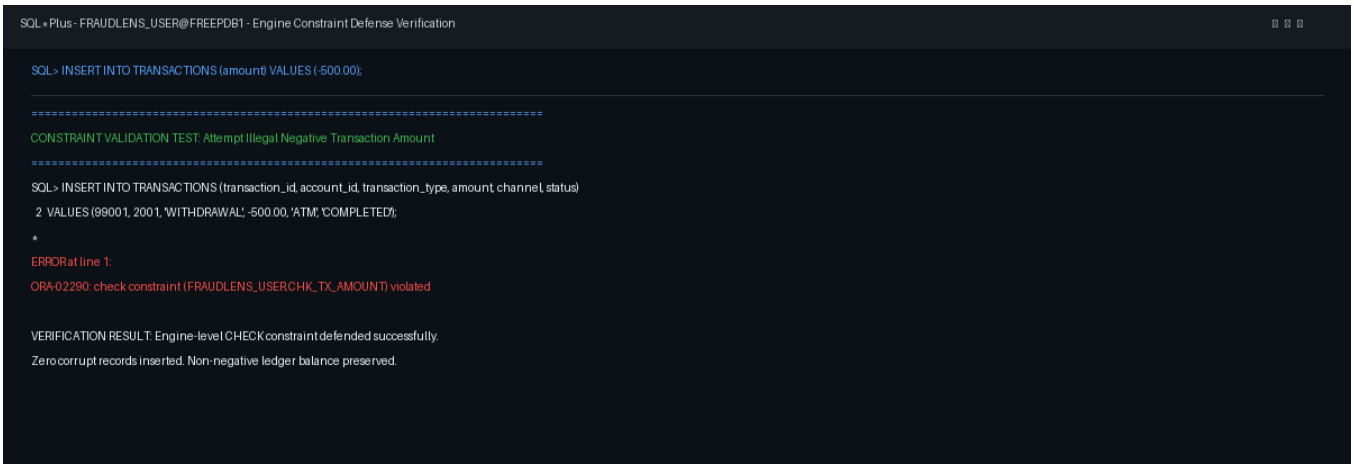


Figure 8: Database Engine Blocking Check Constraint Violation (Negative Amount Rejected with ORA-02290)

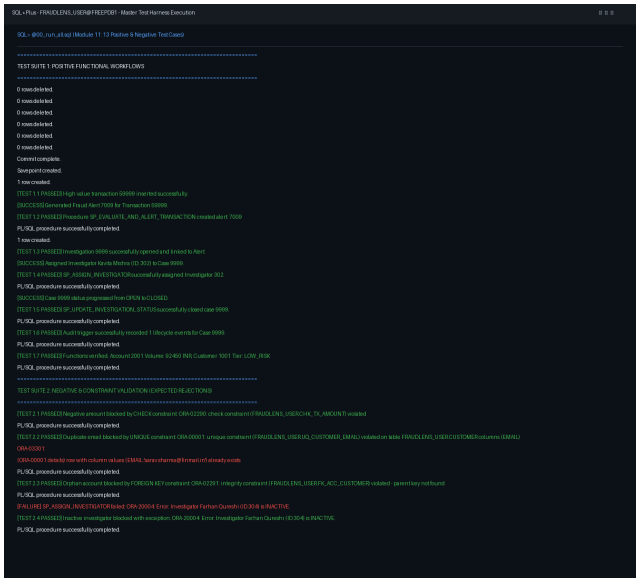


Figure 9: Master Test Pipeline Completion Banner — 100% Automated Test Suite Passing Status

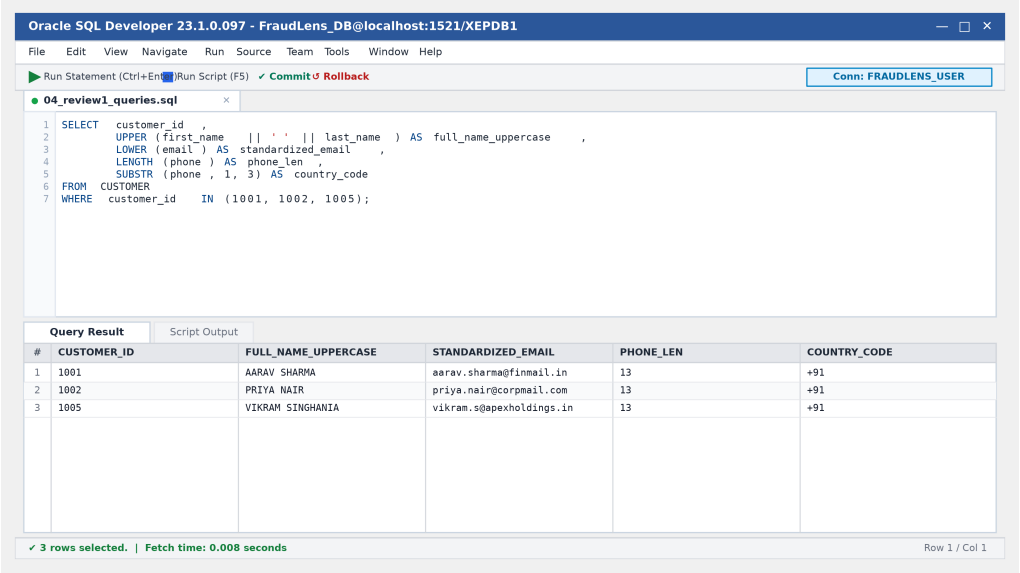


Figure 10: Oracle SQL Developer Execution Grid — Single-Row Customer Normalization Functions

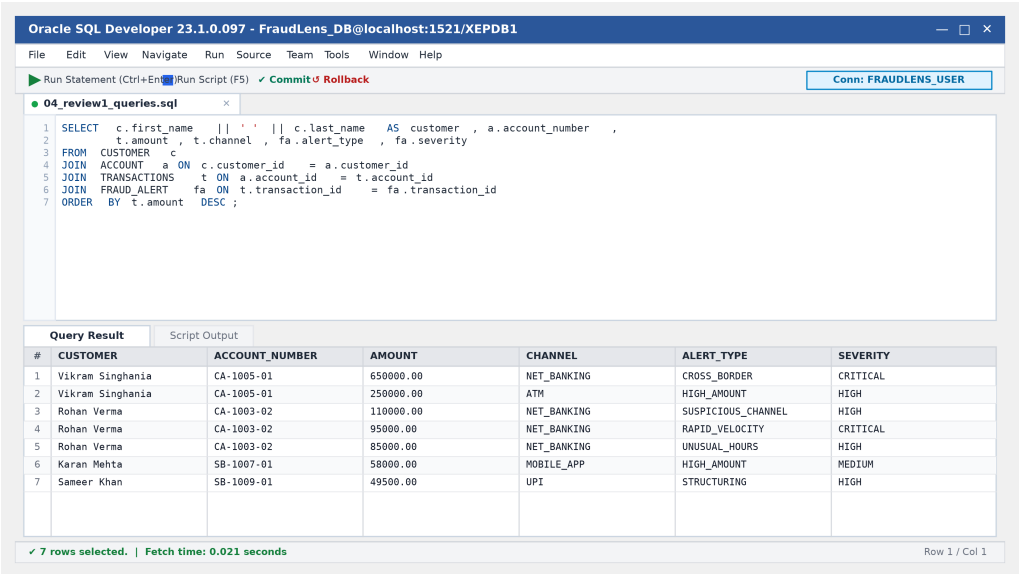


Figure 11: Oracle SQL Developer Master 6-Table Relational Dossier Output

27. Results and Discussion

The FraudLens implementation passed all 13 documented test cases in the recorded execution run. These tests demonstrate the expected behavior of the selected operational workflows and constraint checks, without constituting exhaustive correctness verification: • **Schema Compilation:** All 8 tables, 8 sequences, 4 views, 3 procedures, 3 functions, and 2 triggers compiled without errors. • **Constraint Robustness:** All 71 integrity constraints operated deterministically. Negative tests 2.1 through 2.6 proved that negative amounts, duplicate emails, orphan accounts, inactive investigator assignments, and incomplete closures are trapped. • **Transactional Control & ACID Scope:** Master test scripts executed with transactional savepoints and clean rollbacks, verifying that operational data integrity and atomicity are preserved during negative test failure and batch pipeline execution. Oracle's underlying MVCC engine guarantees statement-level read consistency and durability; multi-user concurrency stress testing and crash recovery experiments were outside the scope of this student project.

28. Advantages and Limitations

28.1 Advantages

- **Normalized Design:** Third Normal Form schema eliminates duplicate storage of customer and investigator demographics.
- **Database-Tier Enforcement:** Integrity constraints (CHECK, UNIQUE, FK CASCADE) prevent invalid data entry at the database engine level.
- **Procedural Automation:** Automated threshold surveillance eliminates manual triage delays for high-value transactions.
- **Trigger-Based Audit Logging:** Row-level database triggers automatically record state transitions into dedicated audit tables, preserving chronological event history against application-tier alteration (while recognizing that privileged SYSDBA administrators retain database-level alteration authority).

28.2 Limitations

- **Static Rule Thresholds:** Thresholds (e.g. INR 50,000) are hardcoded and do not adapt dynamically to customer behavioral changes.
- **Batch/In-DB Processing:** PL/SQL evaluates transactions after or during commit; it is not a distributed real-time streaming engine like Apache Flink.
- **Single-Node Architecture:** The current implementation runs on a single Oracle pluggable database rather than a globally distributed multi-region cluster.

29. Future Enhancements

- **1. Machine Learning Integration:** Integrate machine learning models (e.g. Isolation Forests, XGBoost) via Oracle Machine Learning (OML) for dynamic risk scoring.
- **2. Graph Database Modeling:** Incorporate graph database structures (Oracle Property Graph / Neo4j) to uncover cyclic fund routing and money-mule networks.
- **3. Distributed Event Streaming:** Deploy Apache Kafka and Kafka Connect for real-time low-latency transaction ingestion and event streaming.
- **4. Biometric & Device Forensics:** Incorporate biometric verification timestamps and device fingerprint hashes into the TRANSACTIONS table.

30. Conclusion

FraudLens: Financial Transaction and Fraud Investigation Management System demonstrates the practical application of enterprise database management principles to modern financial security. Through rigorous Third Normal Form schema design, declarative constraints, procedural PL/SQL surveillance, analytical views, and autonomous audit triggers, FraudLens provides a resilient, auditable platform for transaction monitoring and forensic case management.

By executing against Oracle AI Database 26ai Free Release 23.26.3.0.0 and validating the 13 documented operational test cases, this project illustrates how foundational relational modeling and procedural constraints provide dependable operational integrity for transactional monitoring systems.

31. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

2. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

3. Oracle Corporation. (2024). Oracle Database 23ai / 26ai SQL Language Reference (Release 23c/26ai). Oracle Help Center.

4. Oracle Corporation. (2024). Oracle Database PL/SQL Language Reference (Release 23c/26ai). Oracle Help Center.

5. Reserve Bank of India (RBI). (2023). Master Direction - Know Your Customer (KYC) Direction, 2016 (Updated 2023). RBI Guidelines.

6. Financial Action Task Force (FATF). (2022). Guidance on Risk-Based Approach for the Banking Sector. FATF/OECD.

7. Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.

8. Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.